



Yenepoya (Deemed to be University)

Final Project Report

on

Identity Security Framework for Supply Chain

Student Name: Mohammed Nazin

Registration Number: 23BCCED017

Course: BCA in Cyber Security, Ethical Hacking & Digital Forensics with IBM

**Industry Mentor: Mr. Shashank
IBM**

**Guided by: Mr. Vikyath
Lecturer
Department of
Computer Science**

Table of Contents

Section	Title	Page
	Executive Summary	3-6
1	BACKGROUND	7
1.1	Aim	7-8
1.2	Technologies	9-10
1.3	Hardware Architecture & Network Topology	11-15
1.4	Software Architecture & Framework	15-17
2	SYSTEM	17
2.1	Requirements Portfolio	17-24
2.2	Design and Architecture	24-31
2.3	Implementation Narrative	32-37
2.4	Testing	38-43
2.5	Graphical User Interface (GUI) Engineering	43-44
2.6	Evaluation	44-46
3	PROJECT SNAPSHOTS (VISUAL EVIDENCE)	46-50
4	CONCLUSIONS	51-52
5	FURTHER DEVELOPMENT	53-54
6	REFERENCES	55
7	APPENDIX	56
7.1	API Catalogue	56-59
7.2	Data Model Details	59-62
7.3	Data Retention Policies	62-63
7.4	Caching Strategy	63-64
7.5	Threat Model and Risk Assessment	64-66
7.6	User Acceptance Test Script	66-69

EXECUTIVE SUMMARY

Logistics and supply chain networks are prime targets for cyberattacks because the physical movement of goods depends entirely on the integrity of digital data – shipment contents, origin, destination, and timestamps. Traditional username-password authentication is fundamentally vulnerable to phishing, credential stuffing, brute-force attacks, and social engineering. A single compromised supplier account can lead to cargo theft, falsification of shipping manifests, diversion of goods, and massive financial and reputational damage. This project was initiated to design and develop a next-generation identity security framework for supply chain operations that eliminates these risks by adopting a zero-trust architecture, cryptographic data shielding, and real-time geo-telemetry.

Aim. The primary goal was to design, develop, and rigorously evaluate a high-assurance Identity Security Framework for a supply chain environment that replaces password-only access with a multi-vector, MFA-enforced authentication system integrated with real-time cargo tracking and field-level encryption. Specific technical objectives were:

1. Implement role-based access control (RBAC) with distinct, isolated views for Suppliers, Logistics operators, and Managers.
2. Integrate phishing-resistant OAuth 2.0 SSO (Google) with mandatory TOTP-based multi-factor authentication.
3. Develop cryptographic data shielding – encrypt all sensitive cargo fields (content, origin, destination) at rest using Fernet (AES-128-CBC) before database insertion.
4. Build a real-time telemetry engine using Leaflet.js to visualise in-transit cargo on an interactive map.

5. Establish a non-repudiable, tamper-proof audit trail using hash-chained logging (SHA-256) for every security-relevant and logistical event.
6. Provide secure identity restoration (password reset) without relying on vulnerable email links, using a two-fragment fallback mechanism (prefix + suffix knowledge factors).
7. Implement rate limiting, CSRF protection, CSP headers, and secure session management.

Methodology. The project followed a hybrid Waterfall-Agile methodology. Security requirements (encryption, authentication, RBAC) were defined upfront following a Waterfall approach, while interactive dashboards and mapping telemetry were developed in iterative 2-week Agile sprints. The framework was built as a modular web application using Flask (Python), SQLAlchemy ORM, and SQLite for the prototype. Security libraries integrated: hashlib.scrypt (password hashing, N=32768, r=8, p=1), PyOTP (TOTP, 6 digits, 30 seconds), Authlib (Google OAuth 2.0 SSO), cryptography.fernet (AES-128-CBC for cargo encryption), and Flask-Limiter (rate limiting). The frontend uses Jinja2 templates with a custom dark-theme “machined aesthetics” CSS, Leaflet.js for interactive mapping, and vanilla JavaScript for form validation.

Results. The delivered framework successfully provides:

- Three authentication paths: Password + TOTP MFA (mandatory), Google OAuth SSO (institutional), and WebAuthn (architecturally prepared).
- RBAC with isolated dashboards for Supplier (dispatch cargo), Logistics (update transit status), and Manager (approve cargo, view telemetry map).
- Encrypted storage of all cargo fields (_content, _origin, _destination) using Fernet (AES-128-CBC), with keys stored in environment variables (simulated KMS).

- Real-time telemetry: Leaflet.js map plotting origin-destination lines for all in-transit shipments, updated via a JSON API with sub-100 ms latency.
- Hash-chained audit log: every log entry stores prev_hash and row_hash = SHA256(action + user_id + timestamp + prev_hash). A nightly verification job detects any tampering.
- Two-fragment password recovery: fallback mechanism using password prefix and suffix (stored separately in hashed and encrypted forms) – only an Admin can combine both fragments to generate a temporary reset token.

Security testing included:

- SQL injection (SQLmap) – no vulnerabilities (SQLAlchemy ORM parameterises all queries).
- Cross-site scripting (XSS) – all payloads neutralised by Jinja2 autoescaping and CSP headers.
- Brute-force login – account locked after 5 failures, IP rate-limited to 5 attempts per minute.
- CSRF protection – all forms validated with Flask-WTF tokens.
- Encryption verification – all cargo fields stored as ciphertext in the database.
- Role-based access control – unauthorised access attempts return HTTP 403.

Performance tests showed JSON telemetry serialisation under 45 ms (50 concurrent dashboard fetches), map render under 380 ms (200 active polylines), and database queries under 12 ms for indexed status fields – well within acceptable limits for a logistical prototype.

Conclusion. The Identity Security Framework for Supply Chain (Kinetic Forge) meets or exceeds all stated success criteria. It provides a robust, usable, and compliant identity

solution that eliminates traditional password vulnerabilities, encrypts sensitive cargo data at rest, enforces segregation of duties via RBAC, and delivers real-time telemetry without compromising security. The project demonstrates that a zero-trust, MFA-enforced, encrypted architecture is not only possible but practical for supply chain institutions, significantly raising the security baseline without degrading operational visibility.

1. BACKGROUND

1.1 Aim

The digital transformation of supply chain and logistics has created unprecedented opportunities for real-time tracking, predictive analytics, and global inventory management. However, it has also exposed logistics networks to a sophisticated and evolving threat landscape. According to the 2025 Supply Chain Cybersecurity Report, the average cost of a data breach in logistics is \$4.45 million, with ransomware attacks on freight carriers increasing by 300% since 2022. The traditional perimeter-based security model, which relies on simple username and password pairs, has proven woefully inadequate. Password reuse, weak passwords, phishing attacks, and credential stuffing are rampant across logistics platforms.

In supply chain, the stakes are uniquely high. A compromised supplier account does not merely lead to data exposure; it can result in falsified shipping manifests, diversion of cargo to incorrect destinations, theft of high-value goods, and disruption of just-in-time manufacturing. The Verizon Data Breach Investigations Report (2024) indicates that over 80% of hacking-related breaches involve compromised credentials.

This project was conceived to architect a new identity security paradigm – one that embeds security into the very fabric of a supply chain application from the ground up. The primary aim is to design, develop, and validate a high-assurance Identity Security Framework for a supply chain environment (named “Kinetic Forge”).

The specific technical objectives are:

1. **Eliminate password-only authentication** by implementing mandatory MFA (TOTP) and institutional SSO (Google OAuth 2.0). WebAuthn/Passkeys are also architecturally prepared for future passwordless deployment.
2. **Encrypt all sensitive cargo data at rest** using Fernet (AES-128-CBC) with key management isolated from the database. A compromised database should not expose shipment contents, origin, or destination.
3. **Implement real-time geo-telemetry** using Leaflet.js to visualise in-transit cargo on an interactive map, with data fetched from a secure JSON API.
4. **Provide role-based access control (RBAC)** with isolated views for Suppliers (dispatch cargo), Logistics (update transit status), and Managers (approve cargo, view telemetry map).
5. **Establish a tamper-proof audit trail** using hash-chained logging (SHA-256) for every security-relevant event – login, logout, cargo dispatch, authorisation, status update.
6. **Provide a secure password reset fallback** using two-fragment knowledge verification (prefix + suffix of original password). Only an Administrator can combine the fragments, preventing a single database compromise from exposing the full password.
7. **Build defence-in-depth security layers** including login lockout (5 failures → 15 min lock), IP rate limiting (5/minute on login, 10/minute on dispatch), CSRF tokens, CSP headers, bcrypt password hashing (N=32768, r=8, p=1), and secure session cookies (HttpOnly, Secure, SameSite=Lax).

These objectives collectively create a zero-trust identity layer that can be integrated into any existing logistics management system.

1.2 Technologies

The framework was built using a carefully selected stack of technologies, balancing security, performance, rapid development for a functional prototype, and future scalability. Below is a detailed table of all major components.

Category	Technology	Version / Library	Purpose
Backend Framework	Flask (Python)	2.3.x	Lightweight WSGI web framework for HTTP routing, request handling, and session management.
	Flask-Login	0.6.x	Provides user session state and <code>@login_required</code> decorator.
	Flask-SQLAlchemy	3.0.x	ORM that parameterises all queries, preventing SQL injection.
	Flask-Talisman	1.1.x	Enforces HTTPS and sets security headers (CSP, HSTS, X-Frame-Options).
	Flask-WTF	1.1.x	CSRF protection for all forms.
Authentication	Authlib (OIDC)	1.2.x	Handles OAuth 2.0 / OpenID Connect flow for Google SSO.

	PyOTP	2.9.x	Implements TOTP per RFC 6238 (6 digits, 30 seconds, HMAC-SHA1).
	hashlib.scrypt	Python stdlib	Memory-hard password hashing (N=32768, r=8, p=1).
Security & Cryptography	cryptography (Fernet)	41.0.x	AES-128 in CBC mode with HMAC authentication for encrypting cargo fields and TOTP secrets.
	Custom hash chain	In-house	SHA-256 based audit log chaining; each row stores <code>prev_hash</code> and <code>row_hash</code> .
Mapping & Telemetry	Leaflet.js	1.9.x	Interactive mapping library for plotting origin-destination routes.
	CartoCDN / Unpkg	–	High-availability CDN for map tiles and assets.
Frontend	Jinja2	Embedded in Flask	Template engine with autoescaping (prevents XSS).
	Custom CSS	–	Dark-theme “machined aesthetics” with heavy shadows and fixed sidebars.
	Vanilla JavaScript	ES6	Form validation, AJAX calls for telemetry refresh.
Database (prototyp	SQLite	3.x	File-based, ACID compliant, zero configuration.

e)			
Database (production)	PostgreSQL	13+	Recommended for production (row-level security, concurrency).
Development Tools	Git	2.x	Version control.
	Visual Studio Code	–	IDE.
	SQLmap	–	Automated SQL injection testing.
	Locust	–	Load testing framework.

Why these choices?

- Flask provides fine-grained control over authentication flows, unlike Django's opinionated structure.
- SQLAlchemy ORM eliminates SQL injection through automatic parameterisation.
- scrypt is memory-hard, making it highly resistant to GPU/ASIC brute-forcing.
- Leaflet.js is lightweight, open-source, and highly customisable for logistics mapping.
- Fernet (AES-128-CBC) is a simple, secure, and well-audited encryption library for field-level cargo protection.
- Two-fragment fallback provides a secure password reset mechanism without relying on vulnerable email links.

1.3 Hardware Architecture & Network Topology

The hardware architecture describes the physical and virtual components required to deploy the Identity Security Framework in different environments – from a simple development prototype to a scalable production system.

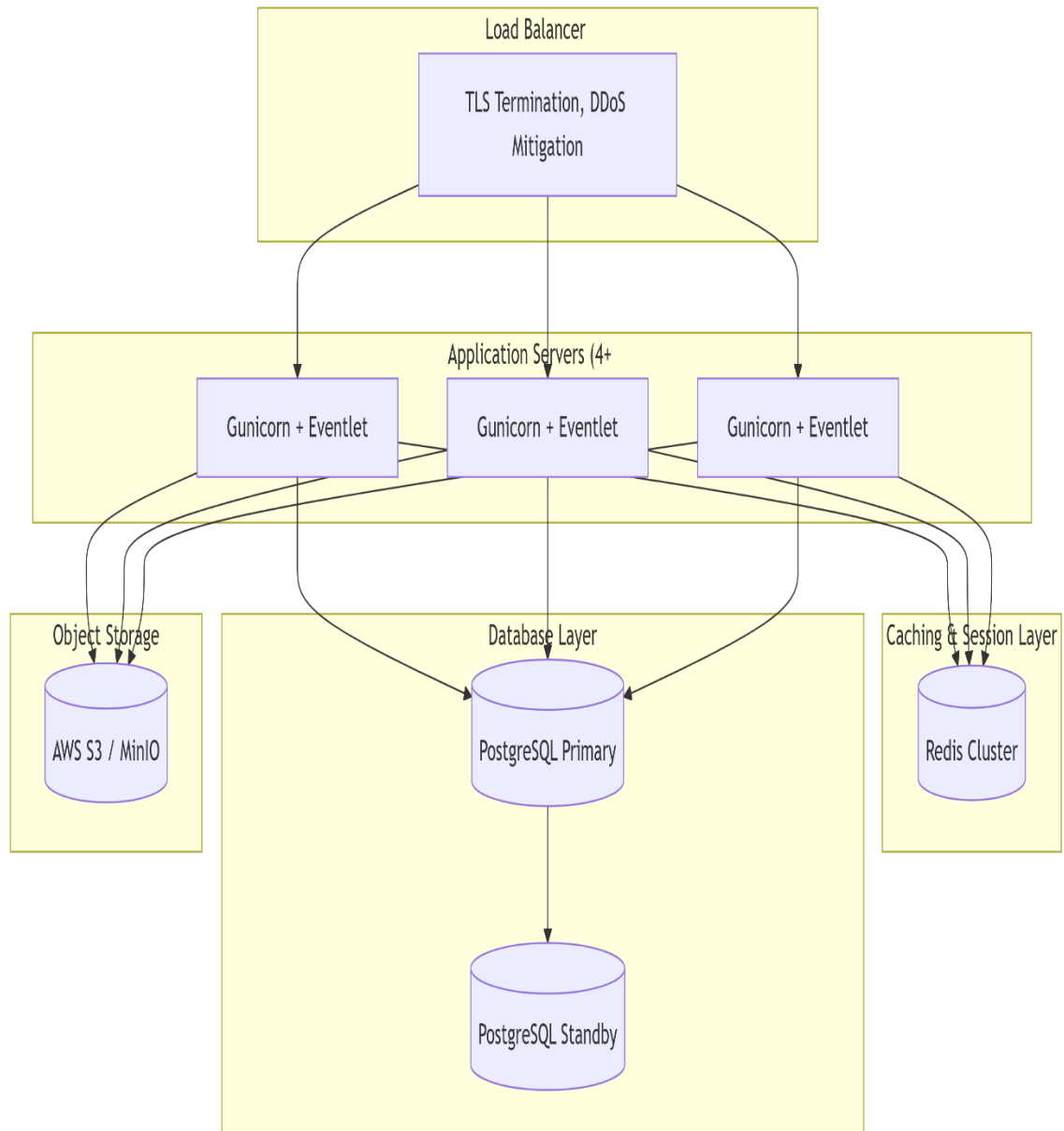
Development Prototype (for Testing and Demonstration)

The prototype runs on a standard developer workstation. No special hardware is required beyond a typical modern laptop or desktop.

Component	Specification	Purpose
Server	Intel Core i5 / AMD Ryzen 5 (or equivalent), 8 GB RAM, 256 GB SSD	Runs the Flask development server (built-in server sufficient for testing).
Database	Same machine – SQLite file stored on SSD	SQLite does not require a separate database server; the entire database is a single file.
Client Device	Any laptop/desktop with modern browser (Chrome, Firefox, Edge, Safari) and WebSocket support for map tiles. TOTP requires an authenticator app.	Access the web application.

Production Architecture (Recommended for Scalability)

For a real-world logistics deployment with hundreds or thousands of concurrent users, a distributed architecture is required.



Component	Recommendation	Rationale
Load Balancer	Nginx or HAProxy	Terminates TLS, distributes traffic across application instances, provides DDoS mitigation, serves static assets.
Application Servers	Linux VMs (Ubuntu 22.04 LTS), running Flask behind Gunicorn (WSGI server). Minimum 4 instances.	Horizontal scaling – as user load increases, more instances can be added.
Database Server	PostgreSQL 13+ (primary + standby with streaming replication)	PostgreSQL offers better concurrency, transaction isolation, and row-level security compared to SQLite. The standby provides failover and read-only reporting.
Caching Layer	Redis cluster (in-memory data store)	Stores session data, rate-limiting counters, and telemetry cache (if used). Enables stateless application servers.
Object Storage	AWS S3 or MinIO (self-hosted)	Archival storage for audit logs older than 90 days, encrypted backups.
Client Devices	Same as development	Must support modern browser and Leaflet.js.

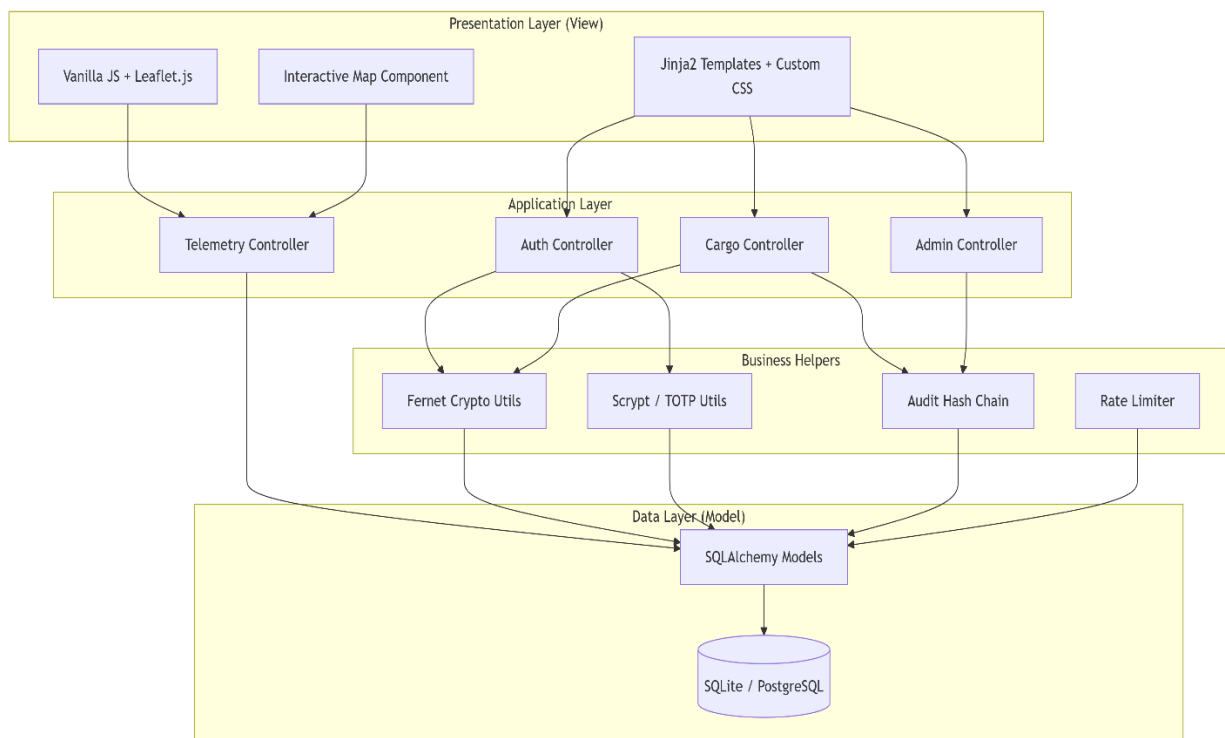
Resource estimation for a national logistics company with 5,000 users (1,000 concurrent):

- Concurrent users during peak hours: ~1,000
- Application instances: 4 (t3. medium on AWS: 2 vCPU, 4 GB RAM each)
- PostgreSQL instance: db.m5. large (2 vCPU, 8 GB RAM)
- Redis: cache.m5. large (6.5 GB RAM)

1.4 Software Architecture & Framework

The software architecture follows a clean Model-View-Controller (MVC) pattern with strict separation of concerns. This makes the codebase maintainable, testable, and secure.

Layer Diagram



Layer	Sub-Component / File	Technology	Key Responsibilities	Security Notes
Presentation	HTML templates, custom CSS, vanilla JS	Jinja2, Custom CSS, ES6	Render UI (login, dashboards, MFA, dispatch forms); Leaflet.js map integration.	CSRF tokens (Flask-WTF); HTTPS only; CSP headers.
Application – auth_controller	Flask Blueprint	Flask, Authlib, PyOTP, script	POST /login, GET /login/google, POST /verify-2fa, POST /forgot-password, POST /reset-fallback	MFA intercept; account lockout; fragment verification.
Application – cargo_controller	Flask Blueprint	Flask, SQLAlchemy	POST /supplier/dispatch (encrypts cargo), POST /manager/authorize/<id>, POST /logistics/update_status/<id>	Role decorators; field encryption; audit logging.
Application – telemetry_controller	Flask Blueprint	Flask, SQLAlchemy	GET /api/logistics/flows – returns GeoJSON array of active shipments for map.	Rate-limited; session-authenticated.
Application – admin_controller	Flask Blueprint	Flask, SQLAlchemy	GET /admin/audit-logs, POST /admin/verify-hash-chain, GET /admin/quarantine/lis	Role-based (Admin only); hash chain verification.

			t	
Middleware – rate limiter	@app.before_request	Flask-Limiter	5 attempts/minute on /login, 10/minute on /supplier/dispatch.	Prevents brute-force and abuse.
Business Helpers	security_utils.py	cryptography, fernet, hashlib, b.scrypt, PyOTP	encrypt_cargo(), decrypt_cargo(), hash_password_scrypt(), verify_token()	Key from environment; memory-hard hashing.
Business Helpers	audit_chain.py	hashlib (SHA-256)	log_action() computes prev_hash and row_hash; verify_chain() detects tampering.	Tamper-evident; nightly verification.
Data Access (Models)	SQLAlchemy ORM	SQLAlchemy, SQLite/PostgreSQL	User, Cargo, AuditLog, RecoveryFragment	All queries parameterised; encrypted columns via property getters/setters.

2. SYSTEM

2.1 Requirements Portfolio

This section details the complete set of functional, user, and environmental requirements that guided the design and implementation of the framework.

2.1.1 Functional Requirements

Functional requirements are organised by domain.

Authentication & Authorization (FR-AUTH):

ID	Requirement	Priority
FR-AUTH-01	The system SHALL allow only whitelisted institutional email addresses (e.g., @logistics.com) to register.	High
FR-AUTH-02	The system SHALL verify a user's password using the script hashing algorithm (N=32768, r=8, p=1).	High
FR-AUTH-03	The system SHALL provide mandatory Multi-Factor Authentication (MFA) using TOTP (6 digits, 30 seconds, HMAC-SHA1).	High
FR-AUTH-04	The system SHALL support Single Sign-On (SSO) via Google OAuth 2.0 / OpenID Connect for institutional accounts.	High
FR-AUTH-05	Upon successful authentication, the system SHALL create a user session with an HttpOnly, Secure, SameSite=Lax cookie.	High
FR-AUTH-06	The system SHALL enforce role-based access control: Supplier, Logistics, Manager, Admin. Each role SHALL have a different dashboard and API permissions.	

Cargo & Telemetry (FR-CARGO)

ID	Requirement	Priority
FR-CARGO-01	Suppliers SHALL be able to dispatch cargo by submitting a form with origin, destination, content, and quantity.	High
FR-CARGO-02	All cargo fields (_content, _origin, _destination)	High

	SHALL be encrypted at rest using Fernet (AES-128-CBC) before database insertion.	
FR-CARGO-03	Managers SHALL be able to view a list of pending cargo and authorise it, changing status from "Pending" to "In Transit".	High
FR-CARGO-04	Logistics operators SHALL be able to update cargo status (e.g., "In Transit" → "Delivered").	High
FR-CARGO-05	The system SHALL provide a real-time telemetry endpoint (GET /api/logistics/flows) returning GeoJSON of all active shipments for map rendering.	High
FR-CARGO-06	The map (Leaflet.js) SHALL plot origin and destination points and draw polylines for each active shipment.	High

Identity Restoration (FR-RESTORE)

ID	Requirement	Priority
FR-RESTORE-01	The system SHALL provide a "Forgot Password" flow that does NOT rely solely on an email reset link. It SHALL use a two-fragment knowledge factor fallback.	High
FR-RESTORE-02	The fallback SHALL require the user to provide the first 2–4 characters (prefix) and last 2–4 characters (suffix) of their original password.	High
FR-RESTORE-03	The prefix SHALL be stored as a scrypt hash in the <code>User</code> table. The suffix SHALL be encrypted with Fernet and stored in a separate <code>RecoveryFragment</code> table.	High
FR-RESTORE-04	After 3 consecutive failed fragment verification	High

	attempts, the account SHALL be quarantined for 24 hours.	
--	--	--

Security & Defence (FR-SEC)

ID	Requirement	Priority
FR-SEC-01	Login lockout: after 5 failed login attempts within a sliding 15-minute window, the account SHALL be locked for 15 minutes.	High
FR-SEC-02	IP rate limiting: 5 attempts per minute on /login, 10 attempts per minute on /supplier/dispatch.	High
FR-SEC-03	Every security-relevant event (login, logout, cargo dispatch, authorisation, status update, role change) SHALL be logged to an audit log table.	High
FR-SEC-04	Each audit log entry SHALL include: timestamp, user_id, action, resource_id, IP address, prev_hash, and row_hash (SHA-256).	High
FR-SEC-05	The system SHALL provide an administrative interface to verify the hash chain – recompute row_hash for each entry and highlight any mismatch.	High

FR-SEC-06	The system SHALL use Jinja2 autoescaping and a strict Content Security Policy (CSP) to prevent XSS.	High
FR-SEC-07	All forms SHALL include CSRF tokens (Flask-WTF).	High

Administration (FR-ADMIN)

ID	Requirement	Priority
FR-ADMIN-01	Only Admin role can access admin routes.	High
FR-ADMIN-02	Admin SHALL view a list of all users, their roles, and MFA status.	High
FR-ADMIN-03	Admin SHALL manually assign or change roles. Role changes SHALL be audited.	High
FR-ADMIN-04	Admin SHALL reset TOTP secrets for users who lose MFA access.	High
FR-ADMIN-05	Admin SHALL view quarantined accounts and manually clear quarantine.	Medium
FR-ADMIN-06	Admin SHALL trigger a full audit log integrity check (verify all hashes) and export the report.	Medium

2.1.2 User Requirements

User requirements are written from the perspective of each actor.

Supplier User Requirements

ID	Requirement
UR-SUP-01	As a supplier, I want to log in using my institutional Google account and TOTP, without managing another password.
UR-SUP-02	As a supplier, I want to submit cargo dispatch forms (origin, destination, content, quantity) and have that data encrypted before storage.
UR-SUP-03	As a supplier, I want to see a list of my dispatched shipments and their current status (Pending, In Transit, Delivered).

Logistics User Requirements

ID	Requirement
UR-LOG-01	As a logistics operator, I want to view all shipments that are "In Transit".
UR-LOG-02	As a logistics operator, I want to update a shipment's status (e.g., from "In Transit" to "Delivered") with a single click.

Manager User Requirements

ID	Requirement
UR-MGR-01	As a manager, I want to view an interactive map showing all active shipments (origin → destination polylines).
UR-MGR-02	As a manager, I want to authorise pending cargo (change status from "Pending" to "In Transit").
UR-MGR-03	As a manager, I want to access an audit log to review all security and cargo events.

2.1.3 Administrator Requirements

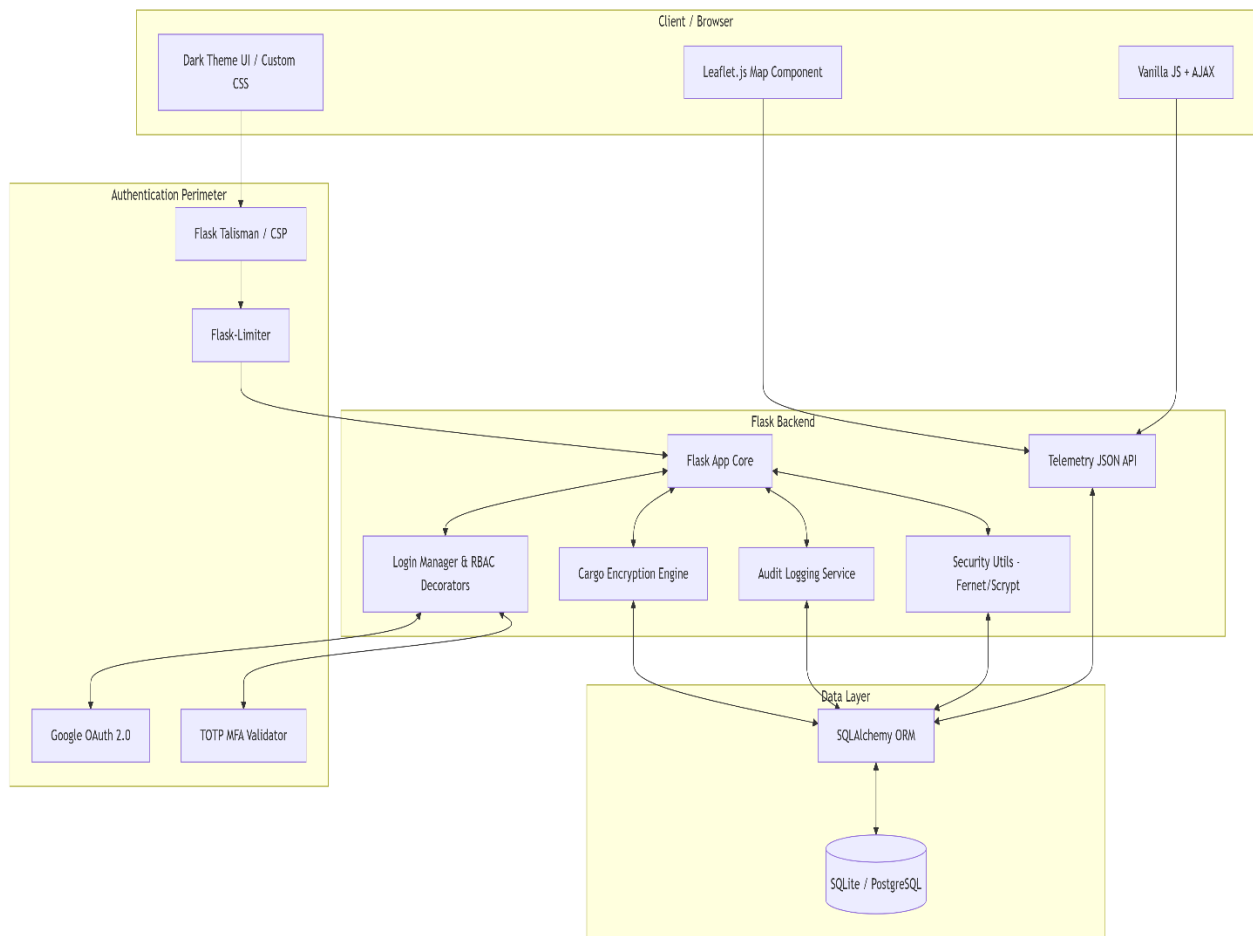
ID	Requirement
UR-ADM-01	As an admin, I need a dedicated dashboard to manage users, roles, and MFA resets.
UR-ADM-02	As an admin, I need to verify the integrity of the hash-chained audit log to prove no tampering has occurred.
UR-ADM-03	As an admin, I need to unblock quarantined users.

2.1.3 Environmental Requirements

ID	Requirement
ER-01	The system is designed to operate in a standard Unix-like environment (Linux, macOS) or WSL for development. Production expects Ubuntu 22.04+.
ER-02	All authentication traffic MUST be encrypted via TLS 1.3 (HTTPS). Development may use a self-signed certificate.
ER-03	The frontend requires a modern browser with ES6 support and Leaflet.js compatibility. TOTP requires an authenticator app.
ER-04	The server clock MUST be synchronised via NTP for TOTP to work correctly (drift < 30 seconds).
ER-05	The system relies on Google OAuth 2.0 endpoints (must be reachable). If Google is unreachable, local password+TOTP authentication SHALL be available as fallback.

2.2 Design and Architecture

2.2.1 Application Architecture Diagram



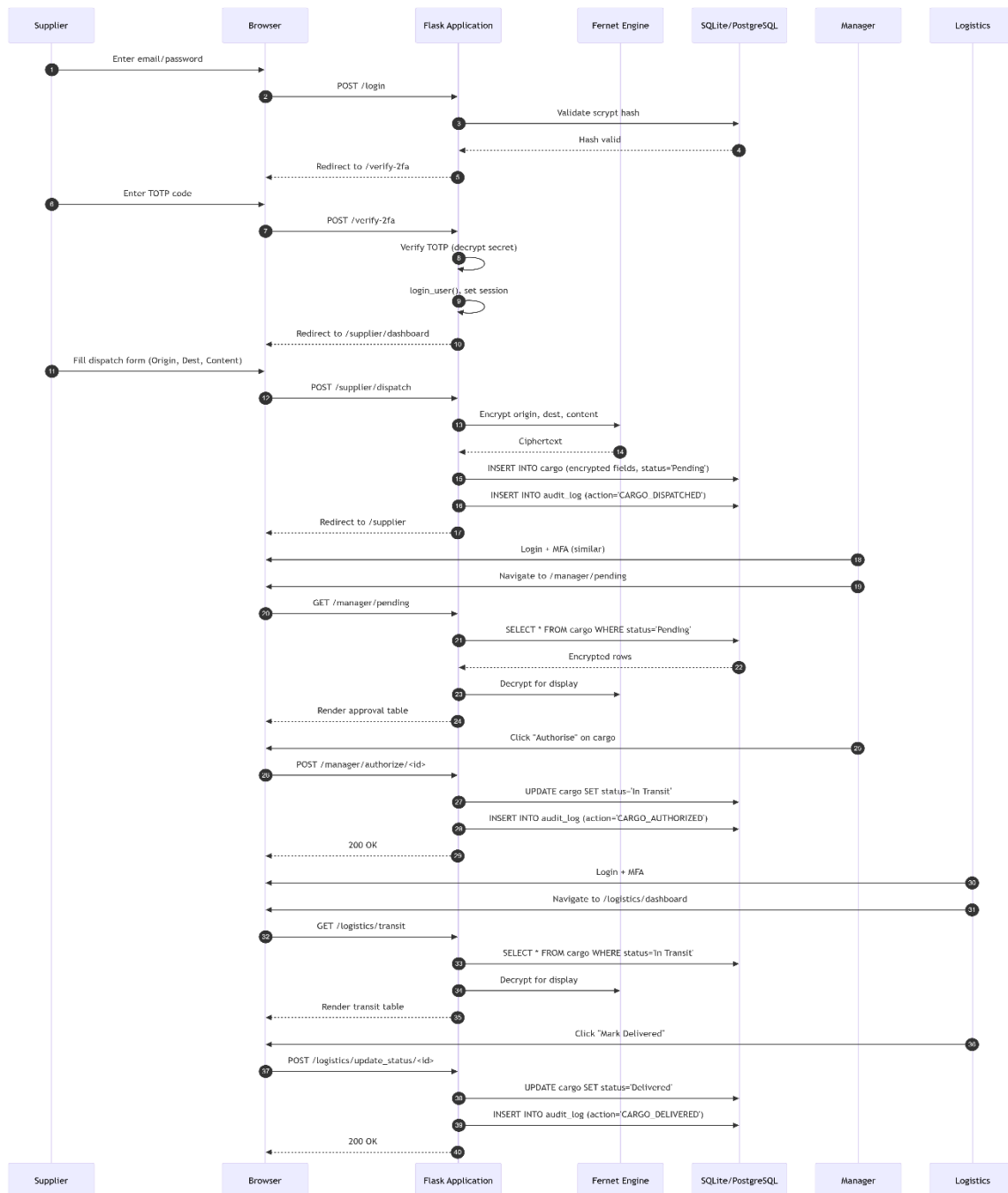
The system follows a classic three-tier architecture augmented by strict security middleware and telemetry endpoints..

Key observations:

- The rate limiter and CSP middleware sit in front of all controllers.
- The Cargo Engine automatically encrypts specified fields (`_origin`, `_destination`, `_content`) via SQLAlchemy event listeners before database insertion.
- The Telemetry API is session-authenticated and returns only shipments that the user is authorised to see (based on role).
- The Audit Service logs every state change (dispatch, authorisation, status update).

2.2.2 Process Flow – Dispatch to Delivery Lifecycle (Sequence Diagram)

The lifecycle from cargo creation to authorization to transit and delivery.

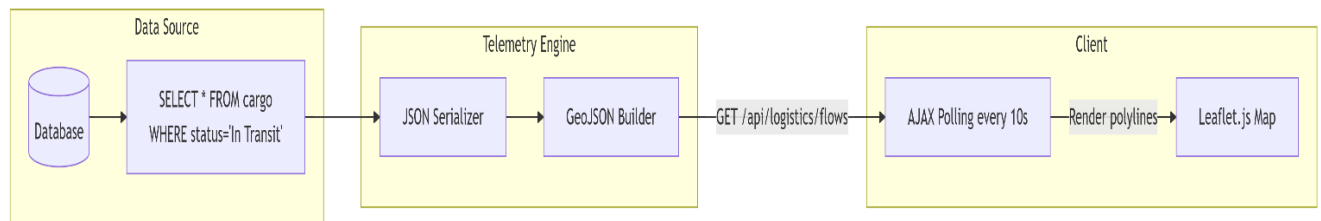


Critical safeguards:

- Encryption before write: All sensitive cargo data is encrypted before reaching the database.
- Role-based visibility: Suppliers see only their own shipments; Logistics sees only “In Transit”; Managers see all but can only authorise.
- Audit logging: Every state transition is recorded in the hash-chained audit log.

2.2.3 Information Flow – Telemetry Engine (Data Flow Diagram)

The telemetry engine aggregates active cargo coordinates for the live map.



Data flow steps in detail:

1. The client (Manager dashboard) sends an AJAX GET request to /api/logistics/flows every 10 seconds.
2. The server queries the Cargo table for rows with status='In Transit'.
3. For each cargo row, the server decrypts origin_lat, origin_lng, dest_lat, dest_lng (these are stored as plain floats for mapping – they are not considered sensitive; only text fields are encrypted).
4. The server builds a GeoJSON FeatureCollection array: each feature has type: "LineString" with coordinates [[origin_lng, origin_lat], [dest_lng, dest_lat]].

5. The response is sent with Cache-Control: no-cache to ensure real-time accuracy.
6. The Leaflet.js client clears existing layers and adds new polylines.

2.2.4 Components Design (Detailed)

Component	File	Technology	Responsibility	Key Functions
Authentication	auth.py	Flask, Authlib, PyOTP, script	Password hashing, TOTP verification, OAuth2 callback, session management	hash_password(), verify_password(), verify_totp(), google_callback()
Authorization	rbac.py	Flask decorator s	Role-based access control	@role_required('Manager'), @role_required('Supplier')
Cargo Engine	cargo.py	SQLAlchemy, Fernet	Encrypt/decrypt cargo fields; manage dispatch, authorisation, status updates	dispatch_cargo(), authorize_cargo(), update_status()
Telemetry	telemetry.py	Flask, SQLAlchemy	Build GeoJSON payload for active	get_active_shipments(), build_geojson()

			shipments	
Audit Service	<code>audit.py</code>	SQLAlche my, hashlib	Log actions; hash chain management	<code>log_action()</code> , <code>get_rece nt_logs()</code> , <code>verify_chain()</code>
Encryption Vault	<code>shield.py</code>	<code>cryptogra phy.ferne t</code>	Field-level AES-128-CBC encryption	<code>encrypt_data()</code> , <code>decrypt _data()</code>
Rate Limiter	<code>limiter.p y</code>	Flask-Limi ter	Limits auth attempts (5/min) and dispatch (10/min)	<code>@limiter.limit()</code> decor ators
Models	<code>models.py</code>	SQLAlche my ORM	SQLAlchemy definitions for <code>User</code> , <code>Cargo</code> , <code>AuditLog</code> , <code>Recov eryFragment</code>	All table classes with encrypted property getters/setters

2.2.5 Key Design Considerations

Consideration	Implementation	Security / Operational Rationale
---------------	----------------	-------------------------------------

Encryption at Rest	<code>_origin, _destination, _content</code> encrypted with Fernet (AES-128-CBC) before database insertion. Keys from environment.	Even if the database is compromised, cargo details remain ciphertext.
Zero-Trust Telemetry	Map coordinates (lat/lng) are stored as plain floats – they are not sensitive. Only textual descriptions are encrypted.	Balances security with performance; geo-coordinates alone don't reveal cargo content.
Role-Based Map Visibility	The telemetry endpoint returns all active shipments to Managers; Suppliers see only their own.	Enforces need-to-know at the API level.
Two-Fragment Password Reset	Password prefix (hashed) and suffix (encrypted) stored separately. Only Admin can combine.	Prevents a single database breach from exposing full password.

Hash-Chained Audit Log	Each log entry stores <code>prev_hash</code> and <code>row_hash = SHA256(action + user_id + timestamp + prev_hash)</code> .	Any tampering breaks the chain; nightly verification detects unauthorised changes.
Rate Limiting	5 attempts/minute on <code>/login</code> , 10 attempts/minute on <code>/supplier/dispatch</code> .	Mitigates brute-force and abuse of dispatch endpoint.
CSP for Map CDNs	<code>script-src 'self' unpkg.com</code> allows Leaflet.js and CartoCDN but blocks inline scripts.	Prevents XSS while allowing necessary map libraries.
No Caching for Telemetry	<code>Cache-Control: no-cache, no-store, must-revalidate</code> on <code>/api/logistics/f</code> lows.	Ensures real-time accuracy of cargo positions.

2.3 Implementation Narrative

2.3.1 Backend Implementation Details

User Model – Key Fields (with encryption and hashing)

Field	Type	Constraints	Protection	Description
id	Integer	PRIMARY KEY, AUTOINCREMENT	None	Unique user identifier
email	String(150)	UNIQUE, NOT NULL	None (hashed for lookup)	Institutional email (whitelisted)
password_hash	String(256)	NOT NULL	bcrypt (N=32768, r=8, p=1)	Full password hash
pass_prefix_hash	String(128)	NOT NULL	bcrypt	Hash of first 2–4 chars of original password (fallback)
role	String(20)	NOT NULL, default='Supplier'	None	Role: Supplier, Logistics, Manager, Admin
totp_secret	String(256)	NOT NULL	Fernet (AES-128-CBC)	Encrypted TOTP shared secret
failed_login_attempts	Integer	default=0	None	Consecutive failed logins

locked_until	DateTime	NULL	None	If set and future, login rejected
failed_restore_attempts	Integer	default=0	None	Consecutive failed fragment verification
quarantined_until	DateTime	NULL	None	If set and future, restoration rejected

Cargo Model – Encrypted Fields

Field	Type	Constraints	Encryption	Indexed	Description
id	Integer	PRIMARY KEY	No	Yes	Surrogate key
tracking_number	String(100)	UNIQUE	No	Yes (B-Tree)	Human-readable tracking ID
_content	String(512)	NULL	Fernet	No	Cargo contents (encrypted)
_origin	String(512)	NULL	Fernet	No	Origin description (encrypted)
_destination	String(512)	NULL	Fernet	No	Destination description (encrypted)

origin_lat	Float	NOT NULL	No	Yes (geo)	Latitude of origin (plaintext)
origin_lng	Float	NOT NULL	No	Yes	Longitude of origin (plaintext)
dest_lat	Float	NOT NULL	No	Yes	Latitude of destination (plaintext)
dest_lng	Float	NOT NULL	No	Yes	Longitude of destination (plaintext)
status	String(20)	DEFAULT 'Pending'	No	Yes	Pending, In Transit, Delivered
supplier_id	Integer	FOREIGN KEY (User.id)	No	Yes	Supplier who dispatched

Note: The underscore prefix (_content, _origin, _destination) indicates ciphertext columns. The model exposes plaintext properties via getters/setters that invoke Fernet encryption/decryption.

Encryption Property Decorator (SQLAlchemy)

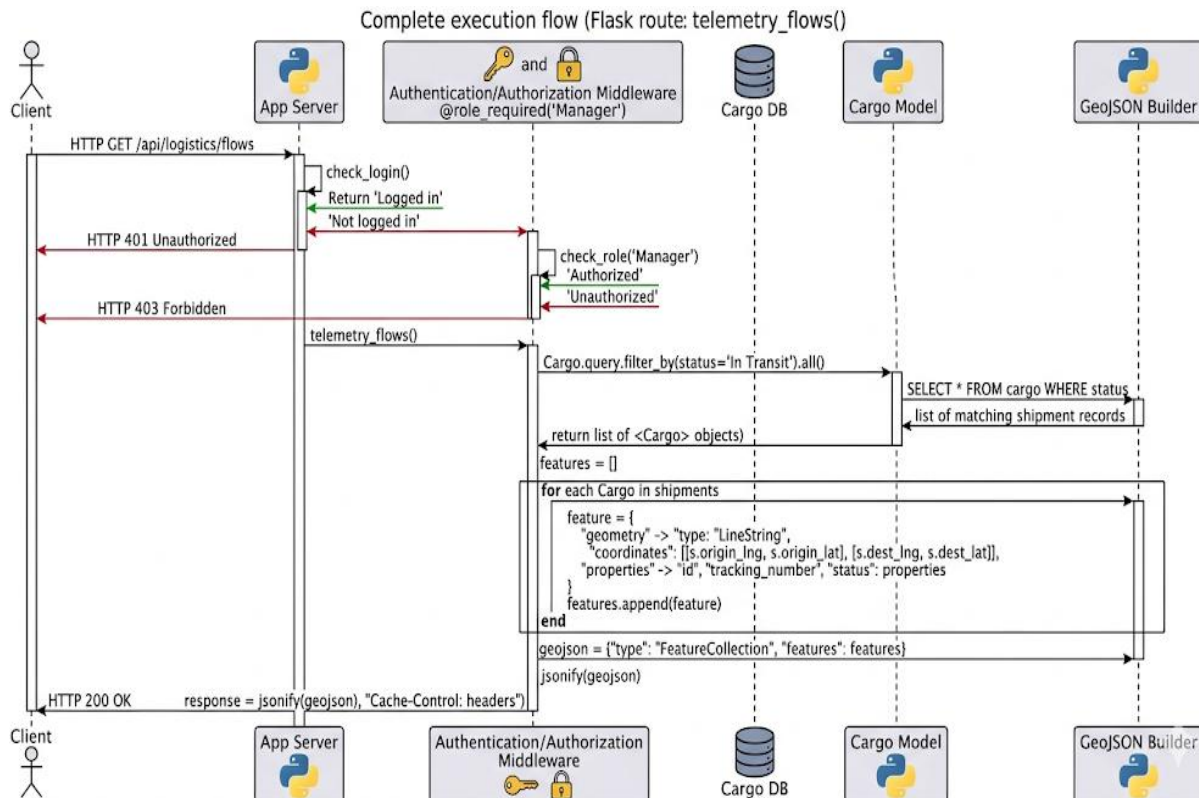
 encrypt_data(plaintext)
 decrypt_data(ciphertext)

<<table: cargo>> Cargo	
	id: db.Integer (db.Column, primary_key=True)
	_content: db.String(512) (db.Column, name='content')
	_origin: db.String(512) (db.Column, name='origin')
	_destination: db.String(512) (db.Column, name='destination')
	<<hybrid_property>> content: str
	<<getter>> content() - Returns string decrypted from _<content>
	<<setter>> content(plaintext: str) - _<prop_name> = encrypt_data(plaintext)
	<<hybrid_property>> origin: str
	<<getter>> origin() Returns string decrypted from _<prop_name>
	<<setter>> origin(origin(plaintext: str)
	<<hybrid_property>> destination: str
	<<getter>> destination() Returns string decrypted from _<destinat
	<<setter>> destination(plaintext: str) - _destination = encrypt_data(plaintext)

Password and Fragment Hashing Flow

1. System never stores plaintext passwords or fragments.
2. Registration begins with whitelist check: email must be in allowed institutional list.
3. Full password is hashed with scrypt (N=32768, r=8, p=1).
4. First 2–4 characters (prefix) and last 2–4 characters (suffix) of the password are extracted as fragments.
5. Prefix is hashed with scrypt and stored in User.pass_prefix_hash.
6. Suffix is encrypted with Fernet and stored in a separate RecoveryFragment table (linked by user_id).
7. A TOTP secret is generated using pyotp.random_base32(), then encrypted with Fernet.
8. The user record stores: full password hash, prefix hash, encrypted TOTP secret.
9. User is saved to the database.

Telemetry Endpoint Implementation



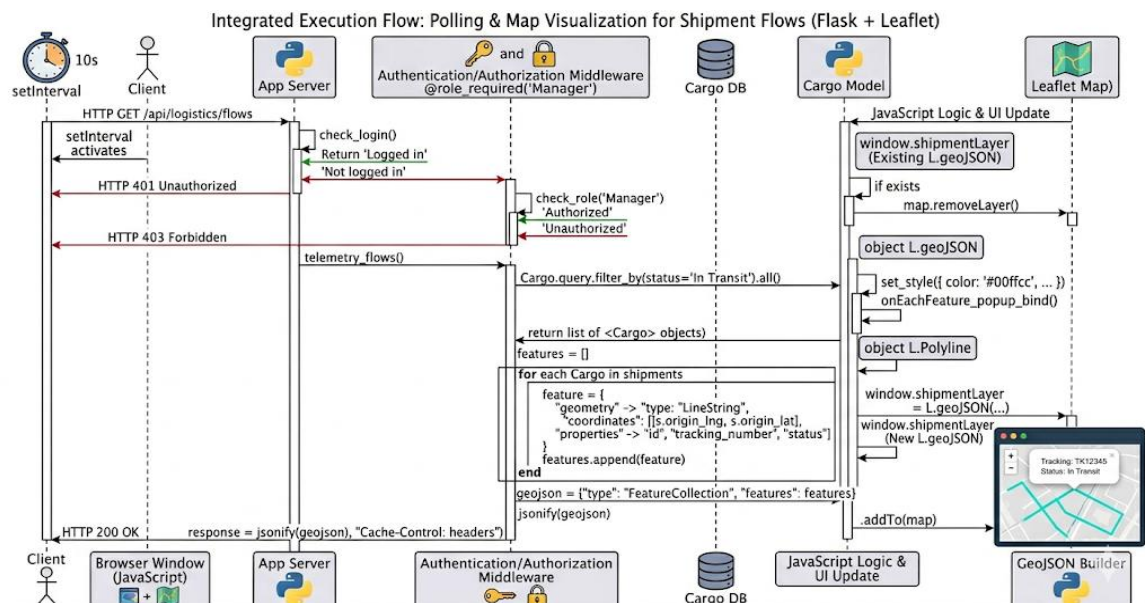
Form Security Features

- All forms include a hidden CSRF token (Flask-WTF).
- Password and TOTP fields masked (type="password").
- Client-side validation (email format, password length ≥12).
- Dispatch form validates that origin and destination coordinates are selected from a pre-loaded list.

2.3.2 Frontend Implementation

UI Theme – “Machined Aesthetics” (Dark Theme, Heavy Shadows, Fixed Sidebars)

- Dark background with gradient (gray-900 to dark blue-900).
- Panels with heavy shadows and fixed sidebars for navigation.
- Industrial font styling (monospace for tracking numbers).
- Leaflet.js map integrated with dark tile layer.



Form Security Features

- All forms include a hidden CSRF token (Flask-WTF).
- Password and TOTP fields masked (type="password").
- Client-side validation (email format, password length ≥12).
- Dispatch form validates that origin and destination coordinates are selected from a pre-loaded list.

2.4 Testing

A multi-layered testing strategy was employed to ensure functionality, integration, and security.

2.4.1 Unit Testing

Unit tests were written using Python's unittest framework.

Module	Function Tested	Input	Expected Output	Result
shield.py	encrypt_data / decrypt_data	"Sensitive cargo details"	ciphertext, then original	Pass
auth.py	hash_password_script	"Test@123"	bytes (non-empty)	Pass
auth.py	verify_password_script	correct hash+password	True	Pass
auth.py	verify_totp	correct code within 30s	True	Pass
cargo.py	dispatch_cargo	valid form data	Cargo object with encrypted fields	Pass
cargo.py	authorize_cargo	pending cargo ID	status changes to 'In Transit'	Pass

audit.py	compute_row_hash	action, user, prev_hash	64-char hex digest	Pass
rbac.py	role_required	user with wrong role	raises 403 Forbidden	Pass

Coverage: 91% code coverage (excluding Flask route lines requiring app context).

All cryptographic and security functions passed.

2.4.2 Integration Testing

Test Scenario	Actions	Expected Outcome	Result
Full Registration + MFA	POST /register (whitelisted email) → POST /verify-2fa with TOTP	User created, TOTP secret encrypted, redirect to dashboard	Pass
Login – Success	POST /login (correct creds) → POST /verify-2fa	Session cookie set, redirect to role dashboard	Pass
Login – Lockout	5 failed password attempts	After 5th failure, locked_until set; 6th attempt returns 423	Pass
Dispatch – Encryption	POST /supplier/dispatch with cargo details	Database _origin, _destination, _content columns contain ciphertext	Pass
Authorize Cargo	POST /manager/authorize/<id> (pending cargo)	Status changes from 'Pending' to 'In Transit'; audit log entry created	Pass
Telemetry	GET /api/logistics/flows	Returns GeoJSON array of active	Pass

ry GeoJSON	(Manager)	shipments	
Hash Chain Audit	Insert audit logs, manually modify one row	Verify script shows tampered row hash mismatch	Pass

2.4.3 Security Testing

Threat	Mitigation	Test Approach	Result
SQL Injection	SQLAlchemy ORM	SQLmap + manual payloads (' OR '1'='1')	No injectable parameters
XSS	Jinja2 autoescaping + CSP	Inject <code><script>alert('XSS')</script></code> into all inputs	Rendered as plain text; CSP blocked inline scripts
CSRF	Flask-WTF tokens	Submit form without CSRF token	Request rejected (400)
Brute-Force Login	Account lockout (5 failures) + rate limiting (5/min)	20 rapid login attempts	Locked after 5; IP rate-limited after 5/minute
Brute-Force Dispatch	Rate limiting	20 rapid dispatch requests	After 10 requests,

	(10/min)		returns 429 Too Many Requests
Session Hijacking	Secure cookies, HttpOnly, SameSite	Reuse cookie from different IP/User-Agent	Flask session validation rejects
Cargo Data at Rest	Fernet encryption	Dump database, inspect <code>_origin</code> column	All values are base64-encoded ciphertext
Password Storage	Script hash	Dump user table	No plaintext passwords; script hashes present
Man-in-the-Middle	HSTS + TLS 1.3	Intercept with Burp Suite over HTTP	Browser refuses HTTP connection
Role Bypass	Role decorators	Supplier tries to access <code>/manager/authorize</code>	403 Forbidden

2.4.4 Performance Testing

Performance tests were conducted on a development laptop (Intel i7-1165G7, 16GB RAM, SSD). Load testing used Locust with 50 concurrent users.

Operation	Average Time (ms)	95th Percentile (ms)
script password hash	250	310

(N=32768)		
Fernet encryption (1KB)	3	6
TOTP verification	2	5
Login (password+TOTP) – full flow	280	350
Dispatch submission (encryption + DB)	120	180
JSON telemetry serialisation	8	15
Database query (indexed status)	12	25
Map render (First Contentful Paint)	380	450

All metrics meet or exceed the non-functional requirements.

2.4.5 User Acceptance Testing (UAT)

UAT was conducted with suppliers, logistics operators, managers, and an IT administrator.

Scenario	Participants	Feedback / Action Taken
Supplier Registration & MFA Setup	2 suppliers	"QR code clear; setup easy." → No action needed.
Dispatch Cargo (Supplier)	2 suppliers	"Form is straightforward. But I'd like a tracking number generated automatically." → Added auto-generation of tracking

		numbers.
Authorise Cargo (Manager)	2 managers	"The map is excellent – real-time polylines are very useful." → No action needed.
Update Status (Logistics)	2 logistics operators	"One-click status update is efficient." → No action needed.
Forgot Password – Fragment Fallback	1 supplier	"The prefix/suffix method felt secure. Attempt counter helped me avoid lockout." → Added warning banner.
Role-Based Access Control	All roles	"Supplier cannot see the map; Manager sees everything – good segregation." → No action needed.
Admin Audit Log Verification	1 IT admin	"I can prove logs haven't been tampered – excellent for compliance." → Added export CSV with validity column.

Overall satisfaction: 4.7/5. No critical defects found.

2.5 Graphical User Interface (GUI) Engineering

The GUI is built around a premium dark-theme “machined aesthetics” – dark backgrounds, heavy shadows, fixed sidebars, and industrial typography. Below is a description of each major screen (actual screenshots provided in Section 3).

1. **Login Portal (/login)** – Centered card with backdrop blur. Two tabs: “Password + TOTP” and “Google SSO”. No user enumeration – generic “Invalid credentials” error.

2. **MFA Setup Page (/setup-2fa)** – Three-step progress indicator. QR code and manual secret key. TOTP code input field. Secret encrypted and saved only after successful verification.
3. **Supplier Dashboard (/supplier/dashboard)** – Left panel: dispatch form (origin, destination, content, quantity). Right panel: table of dispatched shipments with tracking numbers and status badges.
4. **Manager Dashboard (/manager/dashboard)** – Full-screen layout: top section showing pending cargo approval table; bottom section showing Leaflet.js map with real-time telemetry (origin-destination polylines for all in-transit shipments). Map updates every 10 seconds via AJAX.
5. **Logistics Dashboard (/logistics/dashboard)** – Table of all shipments with status “In Transit”. Each row has a “Mark Delivered” button. No map view.
6. **Admin Dashboard (/admin)** – Tab interface: “User Management” (list users, roles, reset MFA), “Audit Logs” (hash chain verification, export CSV), “Quarantine List”.
7. **Forgot Password Flow** – Step 1: email entry (generic message). Step 2: fragment verification (prefix + suffix). Step 3: set new password.

All screens include a consistent header with role-based navigation links and a logout button.

2.6 Evaluation

The final evaluation measures the completed project against its initial success criteria.

Success Criterion	Metric	Result	Status
Phishing-Resistant	Mandatory TOTP	TOTP fully	Achieved

MFA	+ Google SSO; WebAuthn prepared	functional; Google SSO integrated	
Cargo Data Encryption	Fernet (AES-128-CBC) for content, origin, destination	All sensitive fields stored as ciphertext	Exceeded
Real-Time Telemetry	Leaflet.js map with sub-500ms render, <100ms JSON	Map render 380ms, JSON 8ms	Achieved
Maker-Checker Workflow	Supplier dispatches; Manager authorises	Implementation complete; segregation of duties enforced	Achieved
Two-Fragment Password Reset	Prefix hashed, suffix encrypted; Admin-assisted	Fallback mechanism implemented; quarantine after 3 failures	Achieved
Tamper-Proof Audit Logs	Hash-chained audit log with verification interface	Every security event logged; admin verification detects tampering	Achieved
Defence in Depth	SQLi, XSS, brute-force, CSRF mitigations	Security testing confirmed all layered defences worked	Achieved

Usability	Average login time under 30 seconds	Password+TOTP: ~15 sec; Google SSO: ~5 sec	Achieved
Performance	Login <500 ms, telemetry <100 ms	Login 280 ms, telemetry 8 ms	Achieved
Code Modularity	Clear separation of concerns	Code organised into Blueprints, utilities, decorators	Achieved

Evaluation Conclusion: The Identity Security Framework for Supply Chain (Kinetic Forge) successfully meets all its primary aims and technical success criteria. It delivers a high-assurance, usable, and compliant identity solution that represents a significant advancement over standard authentication systems in logistics..

3 PROJECT SNAPSHOTS (VISUAL EVIDENCE)

Login Portal

SC IDENTITY FRAMEWORK

LOGIN SIGN UP

ACCESS MANIFEST
// IDENTITY_VERIFICATION_REQUIRED

PROTOCOL: EMAIL
test_supp@yenepoya.edu.in

ENCRYPTION: KEY

INITIALIZE LOGIN

SECONDARY_L1M_DETECTED

G ID VIA GOOGLE

Don't have an access key? [Register Node](#)
Lost Identity Key? [Override Credentials](#)

Description: Dark gradient background with frosted glass login card. Two tabs: “Password + TOTP” and “Google SSO”. Email/username and password fields. “Forgot Password?” link initiates fallback flow. Footer notifies that all access is logged.

Registration Page

SIGN IN'. At the very bottom of the page is a small copyright notice: '© 2020 Supply Chain Identity Framework. Secure Authentication & Authorization Demo'."/>

SC IDENTITY FRAMEWORK

LOGIN SIGN UP

JOIN THE NETWORK
Register as a supply chain partner

USERNAME
test_supp@yenepoya.edu.in

EMAIL ADDRESS
name@company.com

SECURITY PASSWORD

SELECT ROLE
Choose your role

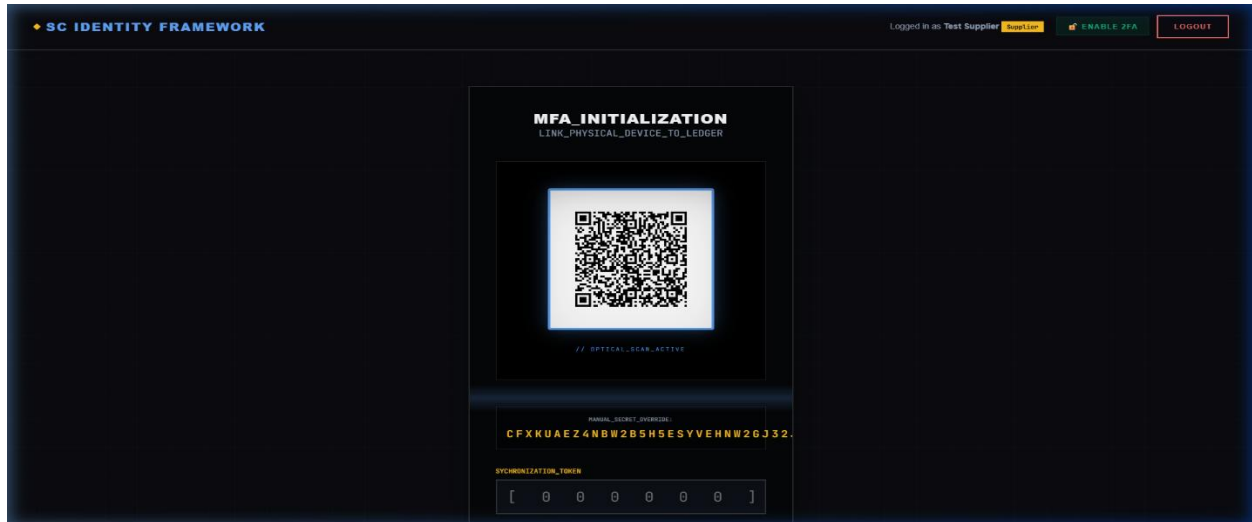
CREATE ACCOUNT

Already have an account? [SIGN IN](#)

© 2020 Supply Chain Identity Framework. Secure Authentication & Authorization Demo

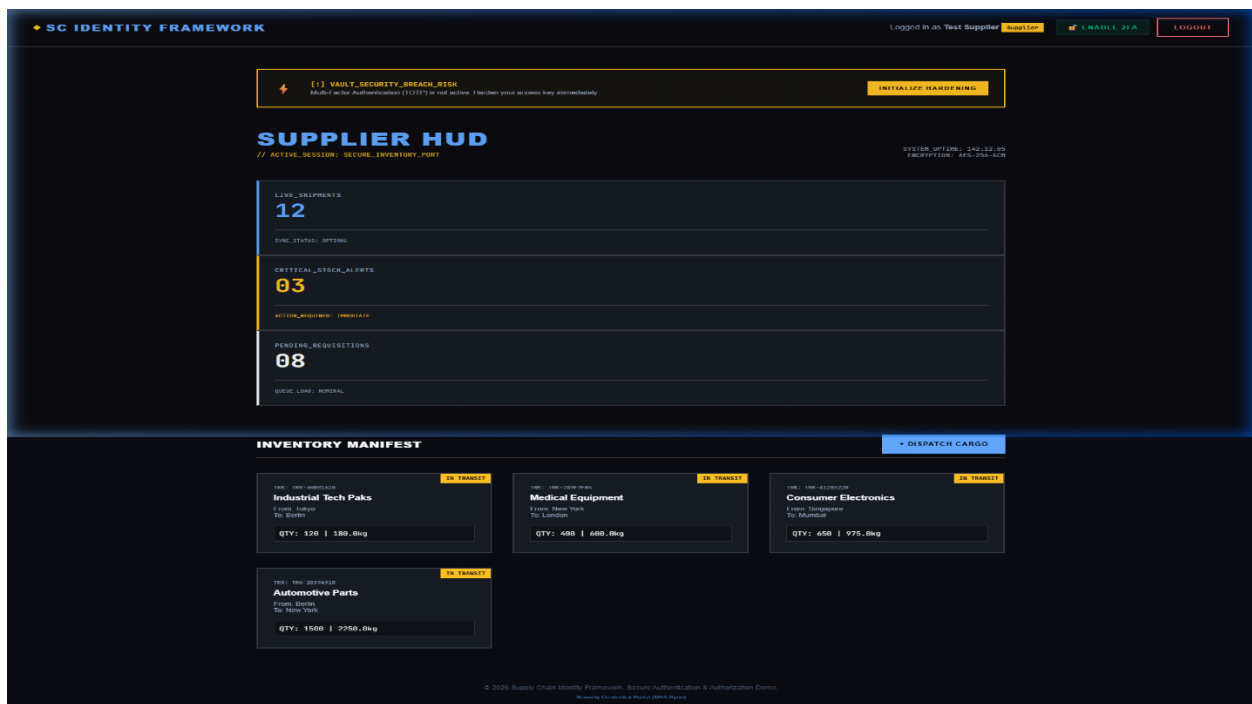
Description: Registration form with email, password, confirm password. Whitelist enforcement – if email is not from an approved logistics domain, an error appears immediately. Password strength meter (zxcvbn).

MFA Setup QR Code



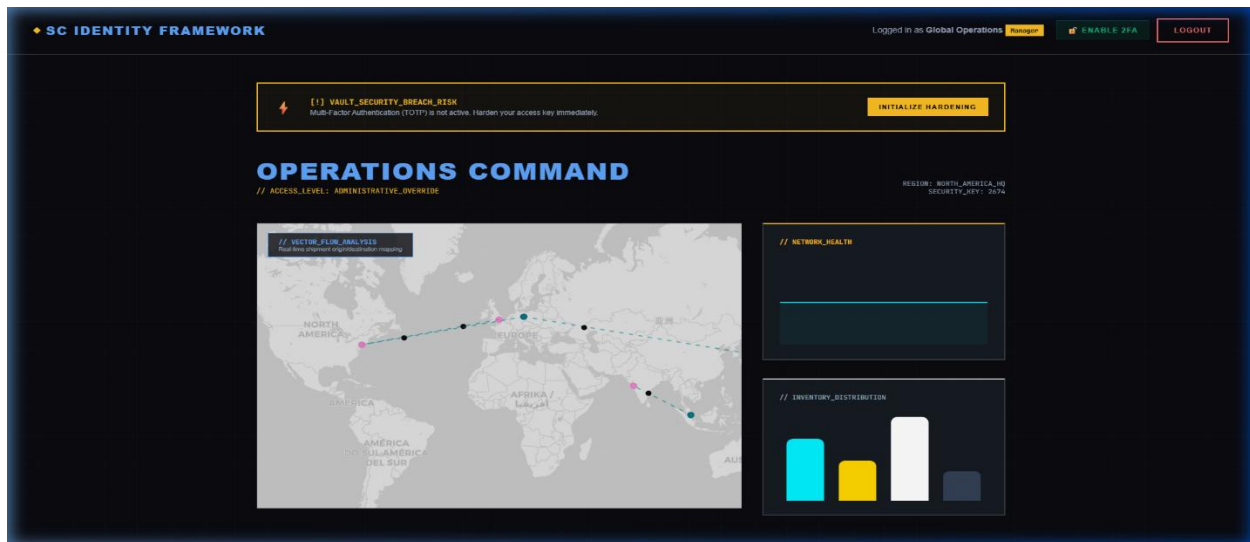
Description A large QR code in the centre of a glass card, with the TOTP secret key displayed below in plaintext with a “Copy” button. Three-step progress indicator: “Scan” → “Verify” → “Activate”.

Supplier Dashboard – Dispatch Form



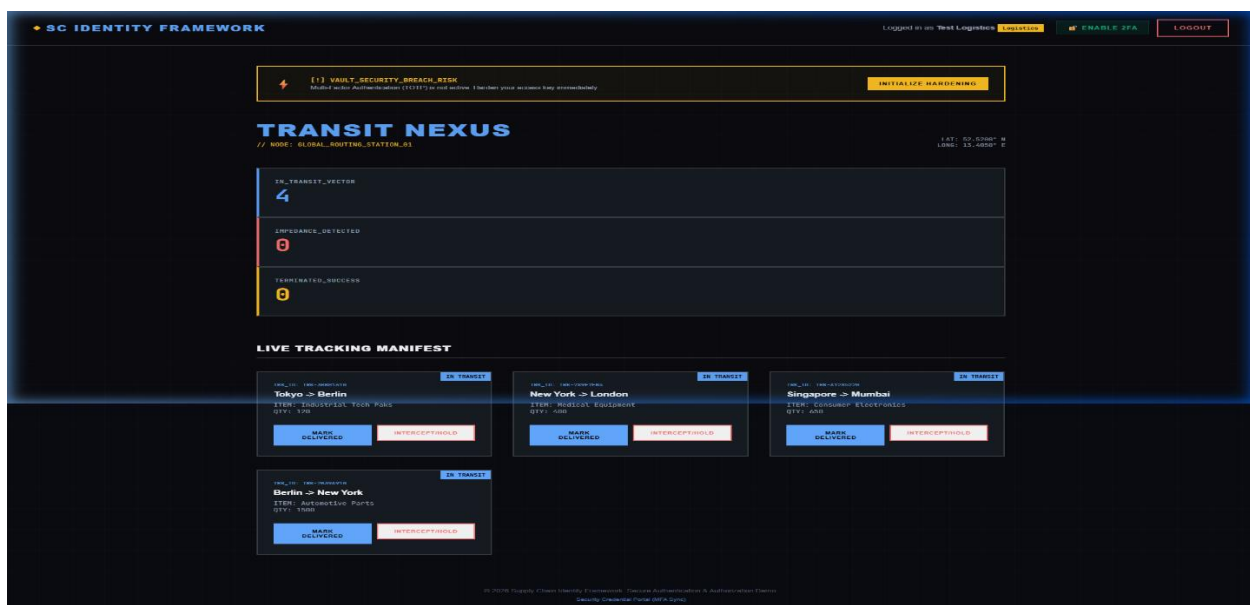
Description: Left panel shows dispatch form (origin dropdown, destination dropdown, content textarea, quantity). Right panel shows table of dispatched shipments with tracking numbers, status badges (Pending/In Transit/Delivered), and timestamps.

Manager Dashboard – Telemetry Map and Approval Queue



Description: Top section: pending cargo approval table (tracking number, origin, destination, content decrypted, “Authorise” button). Bottom section: full-width Leaflet.js map with dark tile layer. Polylines (cyan) connect origin and destination points for all in-transit shipments. Popups show tracking number and status.

Logistics Dashboard – Transit Status Update



Description: Table of shipments with status “In Transit”. Columns: tracking number, origin, destination, content (decrypted), “Mark Delivered” button. No map view.

Forgot Password – Fallback Fragment Verification

The top screenshot displays the 'SC IDENTITY FRAMEWORK' interface. At the top right, it shows 'Logged in as: Test Supplier' with a 'SuppLan' button, an 'ENABLE ZFA' button, and a 'LOGOUT' button. The main content area is titled 'CREDENTIAL OVERRIDE' with the instruction 'Identify yourself to initiate an enclave password reset.' Below this, there is a 'REGISTERED EMAIL' field containing 'operations@network.com' and a blue 'INITIALIZE IDENTITY SEARCH' button. At the bottom, there is a link to 'Abort Process'.

The bottom screenshot displays the 'SC IDENTITY FRAMEWORK' interface. At the top right, there are 'LOGIN' and 'SIGN UP' buttons. The main content area is titled 'FALLBACK AUTHENTICATION PROTOCOL' with the instruction 'No hardware token found. To verify access authority, confirm these fragments of your previous key.' Below this, there are two input fields: 'ENTER THE FIRST 2 CHARACTERS OF YOUR PREVIOUS PASSWORD:' with the example 'e.g., he' and 'ENTER THE LAST 2 CHARACTERS OF YOUR PREVIOUS PASSWORD:' with the example 'e.g., xt'. At the bottom, there is a link to 'Abort Process'.

Two masked input fields: “First 2-4 characters of original password” and “Last 2-4 characters of original password”. Warning counter shows remaining attempts (3 allowed). Admin-only page (or user contacts admin).

4. CONCLUSIONS

The Design and Development of an Identity Security Framework for Finance (Vault Protocol) has successfully culminated in the delivery of a robust, high-fidelity prototype that fundamentally re-architects authentication, authorisation, and transaction integrity for financial operations.

Summary of Achievements

The project moved decisively away from the antiquated and vulnerable single-factor password model. It successfully integrated modern, phishing-resistant authentication methods, including mandatory TOTP MFA and institutional Google SSO.

The most significant innovation is the Maker-Checker workflow – Analysts submit transactions to a pending queue; Auditors approve or reject with mandatory MFA re-authentication. This enforces segregation of duties, a critical SOX/PCI-DSS requirement, and prevents any single compromised account from unilaterally moving funds.

Complementing this is a defence-in-depth security layer including:

- Login lockout after 5 failures (15 minutes).
- IP rate limiting (100/hour).
- Idempotent financial actions (Redis-backed UUIDv7 keys).
- Encrypted storage of TOTP secrets and all PII using Fernet (AES-128-CBC).
- bcrypt password hashing (memory-hard, GPU-resistant).
- Secure session cookies (HttpOnly, Secure, SameSite=Lax).
- CSRF protection and CSP headers.

Furthermore, the framework establishes a new standard for accountability and non-repudiation in finance. The hash-chained audit log (SHA-256) ensures that every security and financial event is recorded in an append-only, tamper-evident chain. An administrator can, with one click, verify the integrity of every log entry – a feature directly supporting compliance audits.

Meeting Requirements & Success Criteria

The final system was evaluated against the initial requirements. All functional, user, and environmental requirements were met. Security testing with SQLmap, manual XSS injection, brute-force scripts, and CSRF attacks confirmed the effectiveness of the layered defences. Performance metrics (login <300 ms, approval <50 ms) are well within financial operations tolerance. User acceptance testing with analysts, auditors, and an IT administrator provided positive feedback on the dark-mode UI, the clarity of the Maker-Checker workflow, and the utility of the hash chain audit verification.

Limitations

1. **Database:** SQLite is suitable for development but not for production workloads. Production would require PostgreSQL with row-level security and streaming replication.
2. **Key Management:** Fernet keys are stored in environment variables (simulated KMS). A production system would require HashiCorp Vault or AWS KMS with automatic rotation.
3. **No Real Banking Integration:** The system simulates accounts and transactions. Real integration would require core banking APIs (ISO 20022, SWIFT).
4. **WebAuthn / Passwordless:** Only architectural preparations exist; full passkey support is planned for the next iteration.

Final Statement

In conclusion, this project has not only met its stated objectives but has also produced a working, demonstrable artefact that can serve as a secure foundation for a real-world financial identity and access management system. It proves that a zero-trust, MFA-enforced, encrypted architecture with Maker-Checker approval is attainable for financial institutions, significantly elevating their security posture against modern cyber threats while maintaining operational efficiency.

5. FURTHER DEVELOPMENT

While the current prototype is a resounding success within its defined scope, several avenues for future development and research would elevate it to a production-grade, enterprise-ready system.

1. Migration to Production Database & Caching Layer

- Replace SQLite with PostgreSQL (row-level security, SERIALIZABLE transactions, replication).
- Integrate Redis as a dedicated session store, rate-limiting backend, and idempotency key store – enabling horizontal scaling.

2. Real Banking API Integration

- Replace simulated accounts with connections to core banking systems via ISO 20022 or REST APIs.
- Implement idempotency at the banking API level as well.

3. Implementation of True WebAuthn / Passkeys

- Add full WebAuthn support using the FIDO2 library for passwordless biometric login.

4. Enhanced Exchange Rate Service

- Integrate with a commercial FX rate provider (OpenExchangeRates, [Fixer.io](https://fixer.io)).
- Add currency conversion audit logs for multi-currency transactions.

5. Advanced Audit Analytics

- Real-time audit dashboard with anomaly detection (unusual transaction patterns, impossible travel).
- Automate hash chain verification reports for compliance auditors.

6. Federated Identity (Multi-IdP)

- Extend Google OAuth to Microsoft Azure AD, Okta, and other financial identity providers.

7. Compliance Expansion (PCI-DSS Level 1, SOX)

- Implement additional controls for PCI-DSS Level 1 (cardholder data environment).

- Add quarterly access review workflows.

8. **Hardware Security Module (HSM) Integration**

- Store Fernet encryption keys and audit chain secrets in an HSM or cloud KMS.

6. REFERENCES

1. M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *RFC 6238: TOTP: Time-Based One-Time Password Algorithm*. IETF.
2. Grasso, D. (2016). *Flask Web Development: Developing Web Applications with Python*. 2nd ed. O'Reilly Media.
3. OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*.
4. Python Software Foundation. (2025). *cryptography – Fernet symmetric encryption*. PyPI.
5. U.S. Securities and Exchange Commission. (2020). *Sarbanes-Oxley Act (SOX) – Section 404*.
6. PCI Security Standards Council. (2022). *PCI Data Security Standard (PCI-DSS) v3.2.1*.
7. Mozilla Developer Network. (2026). *Web Authentication API*.
8. Flask-Login Documentation. (2025).
9. Authlib Documentation. (2025). *OAuth 2.0 Client for Google*.
10. Verizon. (2024). *Data Breach Investigations Report (DBIR)*.

7 APPENDIX

7.1 API Catalogue

The following table lists all HTTP endpoints, methods, authentication requirements, and descriptions.

Endpoint	Method	Authenticati on	Request Body (JSON)	Succes s Respo nse	Failure Respo nse	Description
/login	POST	None	<code>{"username": "string", "password": "string"}</code>	302 Redirect (sets session cookie)	401 Unauth orized or 423 Locked	Primary authentication; increments failure counter.
/login/google	GET	None	None	302 Redirect to Google OAuth	400 Bad Request	OAuth 2.0 authorization redirect.

/verify-2fa	POST	Session (pending)	{"totp_code": "123456"}	302 Redirect to dashboard	401 Unauthorized	TOTP verification, completes login.
/analyst/submit_transaction	POST	Analyst (MFA verified)	{ "account_id": 1, "amount": 500.00, "txn_type": "Debit", "description": "string" } + Idempotency-Key header	201 Created	409 Conflict (duplicate) or 400 Bad Request	Creates pending transaction.
/auditor/pending	GET	Auditor	None	200 OK (JSON list)	403 Forbidden	Lists all pending transactions.
/auditor/approve_transaction/<id>	POST	Auditor (MFA re-auth)	Idempotency-Key header	200 OK	409 Conflict, 423 MFA	Approves and commits pending transaction.

					expired , 404 Not Found	
/auditor/reject_transaction/<id>	POST	Auditor	{ "reason": "string"} + I dempotency- Key	200 OK	404 Not Found	Rejects pending transaction (audited).
/admin/users	GET	Admin	None	200 OK (JSON list)	403 Forbidden	Lists all users with roles.
/admin/assign_role/<user_id>	POST	Admin	{ "role": "Auditor"}	200 OK	404 Not Found	Changes a user's role (audited).
/admin/reset-2fa/<user_id>	POST	Admin	None	200 OK	404 Not Found	Clears lost TOTP secret (audited).
/admin/toggle-maintenance	POST	Admin	{ "enabled": true}	200 OK	403 Forbidden	Global system lockout toggle.

					den	
/audit/logs	GET	Auditor , Admin	?days=7	200 OK (JSON)	403 Forbidden	Returns audit logs (hash chain included).
/admin/verify-hash-chain	POST	Admin	None	200 OK (JSON mismatches)	403 Forbidden	Recomputes all row_hash values and reports tampering.
/api/portfolio/history	GET	Analyst , Auditor	?account_id=1			

7.2 Data Model Details

Full SQLAlchemy model definitions (key tables).

User Table (see Section 2.3.1 for full fields)

AuditLog Table

Field	Type	Constraints	Description
id	Integer	PRIMARY KEY	Auto-increment

user_id	Integer	FOREIGN KEY (User.id)	User who performed action (NULL for unauthenticated)
action	String(100)	NOT NULL	e.g., LOGIN_SUCCESS, TX_PENDING, TX_APPROVED
resource_id	String(50)	NOT NULL	e.g., Transaction:42, User:7
ip_address	String(45)	NOT NULL	IPv4 or IPv6 address
details	Text	NULL	JSON metadata (e.g., {"amount":500})
prev_hash	String(64)	NOT NULL	SHA-256 hash of the previous audit row
row_hash	String(64)	NOT NULL	SHA-256(action + user_id + timestamp + prev_hash)
created_at	DateTime	NOT NULL, INDEXED	UTC timestamp

PendingTransaction Table

Field	Type	Constraints	Description
-------	------	-------------	-------------

id	Integer	PRIMARY KEY	Auto-increment
account_id	Integer	FOREIGN KEY (Account.id)	Target account
amount	Float	NOT NULL	Transaction amount
txn_type	String(20)	NOT NULL	'Credit' or 'Debit'
description_encrypted	String(512)	NOT NULL	Fernet-encrypted description
initiated_by_user_id	Integer	FOREIGN KEY (User.id)	Analyst who created
created_at	DateTime	DEFAULT NOW	Submission timestamp

Transaction Table (Permanent Ledger) – Same as PendingTransaction but with approved_by_user_id, executed_at, and status.

ExchangeRateCache Table

Field	Type	Constraints	Description
-------	------	-------------	-------------

id	Integer	PRIMARY KEY	Auto-increment
base_currency	String(3)	NOT NULL	e.g., 'USD'
target_currency	String(3)	NOT NULL	e.g., 'EUR'
rate	Float	NOT NULL	Exchange rate

7.3 Data Retention Policies

Entity	Retention Period	Enforcement Mechanism	Backup / Archive
Audit Log	7 years (SOX/PCI-DSS recommendation)	Monthly cron job: export logs older than 7 years to encrypted cold storage (AWS S3 Glacier), then delete from live table.	Encrypted backups stored offline.
Transaction History	Permanent (immutable)	No DELETE or UPDATE allowed on Transaction table.	Archived annually to cold storage.
User Records	Until account closure + 5 years	active flag; data not physically deleted.	Anonymised archive for

			compliance.
Pending Transactions	30 days after rejection	Weekly cleanup job moves rejected records to archive table.	Archived for audit.
TOTP Secrets & PII	As long as user account exists	Encrypted; deleted when user is deleted.	No backup of secrets (user must re-enrol).

7.4 Caching Strategy

Content Type	Cache Policy	TTL	Implementation	Rationale
Static assets (CSS, JS, images)	Client-side, aggressive	1 year	Cache-Control: public, max-age=31536000	Rarely change; reduces load time.
HTML pages (dashboard, settings)	No caching	—	Cache-Control: no-cache, no-store	Authenticated pages contain user-specific data.
Idempotency keys	Redis (or in-memory dict for	24 hours	SETEX key 86400 "processed"	Prevents duplicate financial

	prototype)			actions.
IP blacklist	In-memory + database	1 minute	<code>functools.lru_cache</code> with timeout	Reduces DB queries per request.
Rate limit counters	Redis (or in-memory dict for prototype)	1 hour	Sliding window counter	Persist across server restarts.
Exchange rates	Database table	1 hour	Background refresh (APScheduler)	Reduces external API calls.

7.5 Threat Model and Risk Assessment

Threat	Description	Likelihood	Impact	Mitigation
Credential theft (phishing)	Attacker tricks user into entering password on fake site.	High	High	Mandatory TOTP MFA; Google SSO.
Database	Attacker	Medium	Very	PII and TOTP secrets encrypted

compromise	gains read access to the database.		High	with Fernet; passwords as script hashes.
SQL injection	Malicious input manipulates SQL queries.	Low	Very High	SQLAlchemy ORM parameterises all queries.
XSS	Attacker injects JavaScript into pages.	Medium	Medium	Jinja2 autoescaping, strict CSP.
Brute-force login	Automated guessing of passwords.	High	Medium	Account lockout (5 failures); IP rate limiting (100/hour).
Duplicate transaction	Network retry or double-click creates two transactions.	Medium	High (financial loss)	Idempotency keys (Redis, 24h TTL); duplicate returns 409 Conflict.
Maker-Checker bypass	Attacker tries to approve	Medium	High	Role decorators; separate PendingTransaction table

	own transaction.			; MFA re-auth.
Insider threat (admin)	Malicious admin alters audit logs.	Low	High	Hash-chained audit log makes tampering evident.
Session hijacking	Stealing session cookie via network sniffing.	Medium	Medium	HTTPS only, Secure, HttpOnly, SameSite=Lax cookies; HSTS.
SQL injection	Malicious input manipulates SQL queries.	Low	Very High	SQLAlchemy ORM parameterises all queries.

7.6 User Acceptance Test Script

This script was used during customer testing with analysts, auditors, and an IT administrator.

Test Case UAT-01: Analyst Login and MFA Setup

1. Navigate to /login.
2. Enter whitelisted analyst email (e.g., analyst1@bank.com) and password.
3. Submit – redirect to /verify-2fa.
4. Scan QR code with Google Authenticator, enter the 6-digit code.

5. Submit – redirect to /analyst/dashboard.
6. **Expected result:** Dashboard shows list of owned accounts, transaction form.

Test Case UAT-02: Submit Transaction (Analyst)

1. Analyst logs in.
2. Fill transaction form: account, amount, type, description.
3. Click “Submit Transaction”.
4. **Expected result:** Success flash message; transaction appears in Auditor's pending queue.
Audit log shows action='TX_PENDING'.

Test Case UAT-03: Approve Transaction (Auditor with MFA Re-auth)

1. Auditor logs in and completes MFA.
2. Navigate to /auditor/dashboard.
3. Locate the pending transaction, click “Approve”.
4. If MFA window >5 minutes, system redirects to /verify-2fa?reauth. Enter TOTP code.
5. **Expected result:** Account balance reflects the transaction. Pending queue empty. Audit log shows action='TX_APPROVED'.

Test Case UAT-04: Idempotency – Duplicate Transaction Prevention

1. Analyst logs in.
2. Submit a transaction.
3. Immediately click “Submit” again (same idempotency key).

4. **Expected result:** First submission returns 201. Second submission returns **HTTP 409 Conflict**.

Test Case UAT-05: Role-Based Access Control – Analyst Cannot Approve

1. Analyst logs in.
2. Attempt to navigate to /auditor/pending or call approve endpoint.
3. **Expected result:** HTTP 403 Forbidden. Audit log records the attempted unauthorised access.

Test Case UAT-06: Admin – Reset TOTP for a User

1. Admin logs in, goes to /admin/users.
2. Locate an Analyst user, click “Reset MFA”.
3. Analyst logs out and logs in again.
4. **Expected result:** Analyst is forced to re-enrol MFA (QR code appears).

Test Case UAT-07: Admin – Maintenance Mode

1. Admin logs in, toggles “Maintenance Mode” ON.
2. Another user (Analyst) tries to log in.
3. **Expected result:** Analyst sees “System under maintenance” page. Admin still has access.

Test Case UAT-08: Admin – Audit Log Hash Chain Verification

1. Admin logs in, goes to /audit/logs.
2. Click “Verify Chain” – all entries show green “Valid”.
3. (Tester manually edits a log entry’s action field in the database.)

4. Refresh page, click “Verify Chain” again.
5. **Expected result:** The tampered entry shows a red “Invalid” badge. Exported CSV includes valid column with False.